

Research Report: Recent Feature Extractors and Retrieval Pipelines for VPR

Recommendations for the CMU-Africa Night-Time Campus VPR Project

Prepared for the current VPR tutorial codebase

April 24, 2026

Executive Summary

This report surveys recent **primary-source** VPR methods that are realistic candidates for this project. The goal is not to name the absolute best paper in isolation, but to identify the techniques that best match this codebase and project constraints:

- night-time and viewpoint variation,
- campus-scale maps,
- live localization and robot deployment,
- minimal engineering risk for near-term experiments.

Recommended 5 extractors to try in this project:

1. **SALAD (CVPR 2024)**: best first experiment overall. It is a strong single-stage global descriptor, has official Torch Hub support, and fits the current descriptor-based pipeline directly.
2. **BoQ / Bag-of-Queries (CVPR 2024)**: strongest benchmark-oriented follow-up among the direct drop-in descriptor methods, but with much larger descriptor sizes and therefore more expensive search.
3. **AnyLoc-VLAD-DINOv2 (RA-L 2023 / ICRA 2024)**: best general-purpose, no-training baseline and still one of the most valuable transfer-learning experiments for a new campus.
4. **MixVPR (WACV 2023)**: best lightweight and efficient descriptor candidate among the methods surfaced during consolidation, with compact released checkpoints and easy conceptual fit.
5. **VPRTempo (ICRA 2024)**: best robotics-oriented efficiency option. It is not the most direct drop-in replacement for the current pipeline, but it is highly relevant if real-time CPU deployment becomes the priority.

Important nuance: **SelaVPR++ (T-PAMI 2025)** is also a serious candidate and deserves a dedicated section in this report. It is not in the immediate top-5 trial order only because

it introduces a more ambitious **two-stage hashing + reranking pipeline**, which is a larger engineering commitment than the five methods above.

How the Ranking Was Chosen

The ranking is based on the following project-specific criteria:

- **Performance relevance:** robustness to day-night, seasonal, or viewpoint changes.
- **Integration fit:** whether the method naturally plugs into the current pipeline, which expects a global descriptor from `compute_features()` and then runs similarity search.
- **Runtime fit:** whether the method can support a live or robot-facing demo.
- **Code availability:** preference for official code or official pretrained weights.
- **Recency:** emphasis on 2024–2025 methods, while keeping one older but still highly practical baseline where justified.

Why the Current Codebase Matters

Your current implementation is organized around **global descriptors**. In particular:

- `live_vpr/extractors.py` defines the supported descriptors and expects each extractor to return a 2D descriptor matrix.
- `live_vpr/offline.py` builds map descriptors from reference images.
- `live_vpr/online.py` compares a live query descriptor against saved map descriptors.
- `live_vpr/search.py` now supports multiple search backends, so larger descriptors are feasible, but they still increase memory and latency.

This means some new methods are **easy drop-ins** and some are **better treated as separate pipelines**. That distinction is central to the recommendations below.

Comparison with the Gemini List

The Gemini list and this report agree on three methods:

- **SALAD:** both lists identify it as a top-tier modern descriptor.
- **VPRTempo:** both lists identify it as a highly relevant robotics method.
- **AnyLoc:** both lists recognize it as a strong zero-shot baseline.

The main differences are:

- **BoQ** did not appear in the Gemini list, but it should be included because it is one of the strongest recent *drop-in global descriptor* methods with official code and pretrained models.
- **MixVPR** from the Gemini list remains a good recommendation, but I rank it below SALAD, BoQ, and AnyLoc because it is older and less competitive on modern foundation-model-driven VPR than the best 2024 methods.

- **SelaVPR++** from the Gemini list is promising and newer, but I treat it as a *high-upside, higher-effort* option rather than an immediate first-round experiment.

So the consolidated position is:

- keep **SALAD**, **AnyLoc**, **MixVPR**, and **VPRTempo**,
- add **BoQ**,
- keep **SelaVPR++** as a strong notable mention with real follow-up potential.

Ranked Recommendations

1. SALAD (Optimal Transport Aggregation for Visual Place Recognition, CVPR 2024)

What it is. SALAD combines a fine-tuned DINOv2 encoder with an optimal-transport-inspired aggregation layer. The official repository describes two main ideas: using a fine-tuned DINOv2 encoder for richer features, and an aggregation method that extends VLAD with feature-to-cluster and cluster-to-feature assignments plus a dustbin for uninformative features.¹

Why it ranks first for this project.

- It is a **single-stage global descriptor** method, so it matches your current pipeline naturally.
- The official repo provides **Torch Hub** loading, which reduces integration friction.²
- The repo also exposes **multiple descriptor sizes** such as 512+32, 2048+64, and 8192+256, which is useful because you care about both accuracy and live-search cost.³
- It is one of the strongest recent **descriptor-style** methods, not just a reranker or a heavy two-stage system.

Why it is especially relevant to you.

- Night-time robustness is likely to benefit from DINOv2-level features more than older CNN descriptors.
- The compact SALAD variants are attractive for **campus maps plus live search**.
- It can be tested in both `test_campus_dataset_place_level.py` and the live pipeline with minimal architectural changes.

Main caveats.

- DINOv2-based models are still heavier than your current CosPlace/EigenPlaces baselines.
- The official repo was tested with PyTorch 2.1, CUDA 12.1, and xFormers, so environment friction is possible.⁴

Bottom line. If you only try *one* new method first, try **SALAD**. It offers the best balance of modern performance, clean integration, and realistic deployment potential.

¹<https://github.com/serizba/salad>

²<https://github.com/serizba/salad>

³<https://github.com/serizba/salad>

⁴<https://github.com/serizba/salad>

2. BoQ / Bag-of-Queries (CVPR 2024)

What it is. BoQ aggregates local features using a learned bag of queries and cross-attention, then projects the outputs into a global descriptor.⁵

Why it ranks second.

- The official repo provides pretrained **ResNet50+BoQ** and **DINOv2+BoQ** models via **Torch Hub**.⁶
- The released benchmark table is very strong, including **SVOX-night R@1 = 97.7** for the DINOv2-backed model.⁷
- Like SALAD, it produces a **global descriptor**, so it fits your current architecture directly.

Why it is not ranked first.

- The official DINOv2 model uses a **12288-dimensional** descriptor, and the ResNet50 model uses **16384 dimensions**.⁸
- Those descriptor sizes will make search, storage, and reranking more expensive in your live system.
- For a small benchmark this is fine, but for larger maps it creates more engineering pressure than SALAD.

Why it may still be worth trying.

- If your main goal shifts toward **best recall on benchmarks**, BoQ is arguably the strongest direct competitor to SALAD.
- Because you already added configurable ANN search backends, your codebase is better prepared for a high-dimensional descriptor than it was before.

Bottom line. BoQ is probably the best “push for maximum retrieval quality” experiment after SALAD, but it is heavier on the retrieval side.

3. AnyLoc-VLAD-DINOv2 (RA-L 2023 / ICRA 2024)

What it is. AnyLoc is a general-purpose VPR method built around foundation-model features and VLAD aggregation. The project page positions it as a universal VPR system that works across structured and unstructured environments without retraining or fine-tuning, and the repo explicitly presents AnyLoc-VLAD-DINOv2 as the strongest configuration.^{9,10}

Why it ranks third.

- It is the strongest **zero-shot baseline** in the consolidated list.
- It is especially valuable for your project because your campus data differs from standard curated benchmarks, so transferability matters.

⁵<https://github.com/amaralibey/Bag-of-Queries>

⁶<https://github.com/amaralibey/Bag-of-Queries>

⁷<https://github.com/amaralibey/Bag-of-Queries>

⁸<https://github.com/amaralibey/Bag-of-Queries>

⁹<https://anyloc.github.io/>

¹⁰<https://github.com/AnyLoc/AnyLoc>

- It answers a useful scientific question: how far can **general foundation features** go before we need VPR-specific training?

Why it is not ranked higher.

- It is older than SALAD and BoQ.
- The implementation path is heavier than simply loading a single ready-made hub model.
- Compared with SALAD, it is less obviously optimized for **live map retrieval under limited latency**.

Bottom line. If you want the best **general-purpose no-training comparator**, choose **AnyLoc**. It is one of the most informative baselines you can add.

4. MixVPR (WACV 2023)

What it is. MixVPR is an all-MLP feature aggregation method that mixes global feature maps instead of relying on VLAD- or attention-style aggregation. The official repo describes it as a practical, latency-aware VPR technique and releases pretrained checkpoints at **128D**, **512D**, and **4096D** output sizes.¹¹

Why it ranks fourth.

- It is likely the **easiest efficient baseline** to add after SALAD and BoQ.
- It offers compact descriptor sizes that are directly useful for your live map setting.
- It is conceptually simple and pairs well with the search abstractions you already added.

Why it is not ranked higher.

- It predates the strongest 2024 foundation-model-driven methods.
- On pure “latest VPR research” grounds, SALAD, BoQ, and SelaVPR++ are more modern and more ambitious.

Bottom line. MixVPR is a very good **practical engineering trial** even if it is not the most novel method in the list.

5. VPRTempo (ICRA 2024)

What it is. VPRTempo is a temporally encoded **spiking neural network** for visual place recognition. The official project page describes it as an SNN designed for VPR with rapid training and fast query times for compute-limited robotic systems.¹²

Why it makes the top 5.

- It is the most clearly **robotics-first** method in this short list.
- The official page reports **over 50 Hz CPU inference** and support for datasets with **more than 27k unique places**.¹³

¹¹<https://github.com/amaralibey/MixVPR>

¹²<https://vprtempo.github.io/>

¹³<https://vprtempo.github.io/>

- The official package page exposes both a base model and an **quantized int8** variant for lighter deployment.¹⁴

Why it is not ranked higher.

- It is not a clean drop-in global descriptor extractor in the same way SALAD or BoQ are.
- The method is place-centric and training-oriented: each output neuron is meant to respond to a place, so it fits better as a **map- or dataset-specific recognizer** than as a universal descriptor you can compare with cosine similarity.
- In other words, VPRTempo is highly relevant, but it should probably be treated as a **parallel experimental branch**, not just “another descriptor” in `live_vpr/extractors.py`.

Where it fits best in your project.

- A **TurboPi / live robot** branch where CPU speed matters more than leaderboard recall.
- A **map-specific** experimental setup where you train or adapt the model for a given traversal or environment.

Bottom line. If your next goal is **real-time robotics on modest compute**, VPRTempo is one of the most interesting methods to explore. If your next goal is simply to improve your existing descriptor pipeline, start with SALAD or BoQ first.

Other Techniques Worth Watching

SelaVPR++ (T-PAMI 2025)

SelaVPR++ is one of the strongest methods surfaced by the Gemini list and absolutely deserves attention. The official repository presents it as a **T-PAMI 2025** method with Torch Hub support, optional hashing and reranking, and even interchangeable aggregators such as `gem`, `boq`, and `salad`.¹⁵

Why it matters here.

- It is a very serious **efficiency-aware large-scale VPR** method.
- It explicitly supports **binary hashing for first-stage retrieval** and **floating-point reranking**, which aligns with your recent interest in scalable search.
- It is more modern than MixVPR and more retrieval-efficient than raw BoQ-style high-dimensional descriptors.

Why it is a notable mention instead of a first-wave trial.

- To exploit its main strengths, you really want a **two-stage retrieval pipeline**, not just a plain cosine similarity drop-in.
- It is therefore better treated as a **follow-on architecture experiment** after you have tested the easier descriptor-style additions.

¹⁴<https://pypi.org/project/vprtempo/>

¹⁵<https://github.com/Lu-Feng/SelaVPRplusplus>

EffoVPR (ICLR 2025)

EffoVPR is one of the most interesting recent papers because it argues that self-attention features from foundation models can serve as a powerful zero-shot reranker, and that internal ViT layers can produce **compact global features down to 128D**. The paper explicitly claims robustness under **occlusion, day-night transitions, and seasonal variation**.¹⁶

Why it matters here. It directly targets several of your pain points: compact features, strong generalization, and hard appearance changes.

Why it is not in the recommended 5. Based on the primary sources reviewed here, it looks more like an excellent **research watchlist item** than the first method to integrate this week.

SuperVLAD (NeurIPS 2024)

SuperVLAD is presented as a **compact and robust** VPR descriptor built on DINOv2, with optional cross-image encoding.¹⁷

Why it matters here. Its positioning is attractive because compactness and robustness are exactly the tradeoffs you care about.

Why it is not in the recommended 5. Compared with SALAD and BoQ, it looks a bit less battle-tested for immediate drop-in use, and the cross-image encoder adds complexity.

CricaVPR (CVPR 2024)

CricaVPR is a DINOv2-based representation-learning method for VPR with official code and pretrained weights.¹⁸

Why it matters here. It is a solid modern supervised VPR option.

Why it is not in the recommended 5. It is more training-oriented than SALAD and less clearly deployment-oriented than VPRTempo.

Pair-VPR (RA-L 2025)

Pair-VPR is a **two-stage** ViT-based VPR system that first retrieves candidates with a global descriptor and then reranks them.¹⁹

Why it matters here. It is one of the more recent high-performance directions and aligns with your emerging interest in better re-ranking.

Why it is not in the recommended 5. The official repo is explicit that inference is GPU-heavy and training is expensive, making it much less practical for your current live robot timeline.²⁰

Recommended Trial Order

If the goal is to improve the current codebase with minimal disruption:

¹⁶<https://arxiv.org/abs/2405.18065>

¹⁷<https://github.com/lu-feng/SuperVLAD>

¹⁸<https://github.com/Lu-Feng/CricaVPR>

¹⁹<https://github.com/csiro-robotics/Pair-VPR>

²⁰<https://github.com/csiro-robotics/Pair-VPR>

1. **SALAD**
2. **MixVPR**
3. **BoQ**
4. **AnyLoc-VLAD-DINOv2**
5. **VPRTempo**

If the goal is to improve the live robot story even at the cost of a separate code path:

1. **SALAD**
2. **VPRTempo**
3. **MixVPR**
4. **BoQ**
5. **AnyLoc-VLAD-DINOv2**

If the goal is to attempt the most ambitious scalable-search branch after the first wave:

1. **SelaVPR++**
2. **EffoVPR**
3. **SuperVLAD**

Concrete Recommendations for This Repo

Phase 1: easiest drop-in experiments

- Add **SALAD**, **MixVPR**, and **BoQ** as new extractor wrappers under `feature_extraction/` and register them in `live_vpr/extractors.py`.
- Evaluate them first with:
 - `test_campus_dataset_place_level.py`
 - `test_campus_dataset.py`
 - your live map / live localization path
- Because **BoQ** descriptors are large, prefer the new ANN backends in `live_vpr/search.py` when testing live performance.

Phase 2: general-purpose foundation-model baseline

- Try **AnyLoc-VLAD-DINOv2** if you want to know how far a no-retraining foundation-feature pipeline can go on your campus.
- This is especially useful if you want a clean comparison against your manually relabeled place-level dataset.

Phase 3: dedicated robotics branch

- Explore **VPRTempo** as a separate experiment, especially for the TurboPi scenario.
- Do not force it into the current cosine-similarity descriptor pipeline unless you first verify that the package exposes a reusable embedding representation in the way your current code expects.
- A better framing is: **current pipeline = descriptor retrieval**, **VPRTempo branch = direct place recognizer for resource-constrained deployment**.

Phase 4: scalable two-stage retrieval branch

- Explore **SelaVPR++** if you want to extend the architecture toward binary-first retrieval plus floating-point reranking.
- This is the most natural follow-up if your map sizes grow enough that **ANN alone is not the whole story**.

Final Recommendation

If your goal is **one strong next method to try in this repo**, choose **SALAD**. If you want the most balanced five-method trial plan, use this order:

1. **SALAD**
2. **BoQ**
3. **AnyLoc-VLAD-DINOv2**
4. **MixVPR**
5. **VPRTempo**

In short:

- **Best first practical upgrade: SALAD**
- **Best high-performance follow-up: BoQ**
- **Best no-training transfer baseline: AnyLoc**
- **Best compact practical baseline: MixVPR**
- **Best edge/robotics research direction: VPRTempo**
- **Best ambitious scalable-search follow-on: SelaVPR++**

Primary Sources Used

1. SALAD official repository: <https://github.com/serizba/salad>
2. BoQ official repository: <https://github.com/amaralibey/Bag-of-Queries>
3. MixVPR official repository: <https://github.com/amaralibey/MixVPR>
4. VPRTempo official project page: <https://vprtempo.github.io/>

5. VPRTempo official package page: <https://pypi.org/project/vprtempo/>
6. AnyLoc official project page: <https://anyloc.github.io/>
7. AnyLoc official repository: <https://github.com/AnyLoc/AnyLoc>
8. SelaVPR++ official repository: <https://github.com/Lu-Feng/SelaVPRplusplus>
9. EffoVPR arXiv page: <https://arxiv.org/abs/2405.18065>
10. CricaVPR official repository: <https://github.com/Lu-Feng/CricaVPR>
11. SuperVLAD official repository: <https://github.com/lu-feng/SuperVLAD>
12. Pair-VPR official repository: <https://github.com/csiro-robotics/Pair-VPR>